CRU-IAS

Crime Response Unit Intelligence & Analytics System

System Documentation

Prepared by	Priyanka Lenka
Role	Data Analyst — Training Project
Date	April 2026
Version	V2.0
Tools Used	Python, Pandas, Jupyter Notebook, HTML, JavaScript, Chart.js
GitHub	github.com/PRIYANKALENKA07/CRU-IAS

1. Project Overview

1.1 What is CRU-IAS?

CRU-IAS (Crime Response Unit Intelligence & Analytics System) is an end-to-end crime data intelligence system built on FIR (First Information Report) level data collected by the Crime Response Unit (CRU) across 45 districts and 50 police stations in Karnataka, spanning the years 2020 to 2025 covering approximately 2,11,863 registered cases.

The system was designed to solve a very specific and real operational problem: crime data was scattered across 7 raw tables crime records, disposal records, law sections, and more with no unified view. Answering even a simple question like "how many cyber crime cases are pending in a particular police station this year?" required manually joining multiple tables and writing custom queries every single time.

CRU-IAS eliminates that friction entirely. It takes those 7 raw tables, cleans and joins them using a Python-based data pipeline (built with Pandas), and produces a single structured summary dataset. That dataset powers an interactive HTML dashboard built with JavaScript and Chart.js where any user can select a police station, a year, and a crime type, and instantly see total cases, disposed cases, and pending cases, along with a visual chart. No SQL. No manual joins. No waiting.

1.2 Why It Was Built

Before CRU-IAS existed, the Crime Response Unit had a Power BI dashboard that visualized crime data at a surface level it was useful for reporting and presenting numbers to leadership, but it had a fundamental limitation: it was a **presentation layer**, not an **intelligence layer**.

The Power BI dashboard could show you charts and trends, but it could not answer operational questions quickly without someone first preparing and cleaning the data manually behind the scenes. Every new question required a fresh data pull, a fresh join, and a fresh calculation. This made it slow, analyst-dependent, and impractical for day-to-day decision making at the station or district level.

CRU-IAS was built to go one level deeper to handle the **data engineering problem first**, so that the dashboard on top of it is always ready, always accurate, and always fast. The core idea is simple: clean the data once, structure it well, and let the system answer questions automatically from that point forward.

This is why CRU-IAS is not just a dashboard it is a complete pipeline. The Python layer handles all the complexity (cleaning, joining, aggregating), and the HTML dashboard is simply the window through which users interact with the result. Anyone even someone with zero technical background can open the dashboard and get answers in seconds.

The 91% pending case rate discovered in this dataset further reinforced why such a system matters. When nearly 9 out of 10 cases are unresolved, police leadership needs a fast, reliable way to identify which stations have the highest backlogs, which crime types are accumulating the most pending cases, and where resources need to be redirected urgently. CRU-IAS makes that visibility possible.

1.3 Target Audience

CRU-IAS is designed to serve multiple layers of users within and around the law enforcement ecosystem:

Police Leadership (SP, DGP, Range IGP): Senior officers who need a district-wide or state-wide view of case pendency, disposal rates, and crime type distribution without waiting for a manual report to be prepared. CRU-IAS gives them that view on demand.

Station House Officers (SHOs) and District Analysts: Ground-level officers and analysts who need to track the status of cases within their own police station how many are pending, how many were disposed this year, and which crime categories are growing. This is the most frequent use case the system was designed for.

Crime Analysts and Data Teams: Technical users within the CRU who need clean, structured data for deeper analysis, reporting, or feeding into future predictive models. The pipeline output (clean summary CSV) is directly usable for any downstream analysis.

Judicial and Administrative Bodies: Bodies that oversee case disposal timelines and want to understand systemic delays across districts or crime types, particularly for high-priority IPC sections.

Future Developers and Researchers: Anyone who wants to extend this system — add new visualizations, integrate new data sources, or build predictive models on top of the existing pipeline. The architecture is documented in detail in this document for exactly that purpose.

1.4 System Phases

Phase	Name	Description	Status
Phase 1	Data Pipeline	Clean, join & aggregate raw FIR tables into analysis-ready CSV	✓ Done
Phase 2	Analytics Dashboard	Interactive HTML system with dropdowns and Chart.js visualizations	In Progress
Phase 3	Predictive Model	ML-based crime forecasting and risk prediction system	Planned

2. Data Overview

2.1 Data Sources

This system is built on case-level FIR data provided by the Crime Records Unit, spanning January 2020 to January 2025 approximately 5 years of data across 45 districts and 50 police stations.

Table	Records	Columns	Description
crime.csv	2,11,863	18	Core FIR details — location, date, crime type, FIR stage, unit ID
dispose.csv	17,660	5	Case disposal information — disposal date and type
act_section.csv	3,49,029	6	Law sections under which cases are registered — crime classification
ps_unit.csv	1,856	4	Police station master data — unit names and coordinates

accused.csv	2,37,773	15	Demographic details of accused persons
victim.csv	1,41,398	15	Demographic details of victims
chargesheet.csv	54,325	6	Chargesheet filing details and dates

2.2 Key Statistics

1,40,585	12,935	1,27,650
Unique FIRs	Disposed Cases (~9%)	Pending Cases (~91%)

3. Data Pipeline — Phase 1

The data pipeline transforms raw source tables into a clean, aggregated CSV file that powers the analytics dashboard. All steps are implemented in Python using the Pandas library and documented in the Jupyter Notebook: notebooks/01_data_pipeline.ipynb.

3.1 Step 1 — Load & Clean Raw Tables

What was done

Seven raw **CSV files** were loaded into Pandas **DataFrames** at the start of the pipeline: **Crime.csv, Dispose.csv, Act_Section.csv, Accused.csv, Chargesheet.csv, PS_Unit.csv, and Victim.csv**. Of these, the three core tables actively used in the pipeline are **Crime, Dispose, and Act_Section** the remaining four (**Accused, Chargesheet, PS_Unit, Victim**) were loaded for completeness and potential future use.

Once loaded, the Crime table went through the following cleaning steps:

- Dropped junk columns — Unnamed: 18 and Unnamed: 19 were removed using drop(columns=..., errors='ignore').
- Standardized column name — **Unit_ID** was renamed to **Unit_id** to match the casing used in the Dispose and Act tables, which is required for the join key to work correctly.
- Converted date columns to datetime format — FIR_Date in the Crime table and Dispose_Date in the Dispose table were both converted using pd.to_datetime(..., errors='coerce').
- Extracted date components — Three new columns were derived from FIR_Date: FIR_Year (the calendar year), FIR_Month (integer 1–12), and FIR_Month_Name (full month name like "January") using .dt.year, .dt.month, and .dt.strftime('%B') respectively.

Why it was done

- **The Unnamed: 18 and Unnamed: 19 columns were empty artifact columns — they appear automatically when a CSV is exported from Excel with trailing commas or extra whitespace in the last columns. They contain no data and would pollute the DataFrame with null columns if left in.**

- **The Unit_ID → Unit_id rename was not cosmetic. The join key created later (Fir_id + Unit_id) had to be consistent across all three tables. A column name mismatch would silently cause the join to fail or produce wrong results.**
- **Date conversion was necessary because raw CSVs store dates as plain text strings. As strings, you cannot extract the year, filter by month, or do any time-based analysis. Converting to datetime64 unlocks all of Pandas' date functionality.**
- **errors='coerce' was used deliberately — if any date value is malformed or missing, it becomes NaT (Not a Time) instead of crashing the pipeline. This makes the pipeline robust to dirty data.**
- Extracting FIR_Year, FIR_Month, and FIR_Month_Name upfront means every downstream step and the final summary can group and filter by time without recalculating these values repeatedly.

Result

Crime table cleaned from its original shape, with Unnamed: 18 and Unnamed: 19 removed. FIR_Date confirmed as datetime64 dtype with 0 null values after coercion. Three new time columns (FIR_Year, FIR_Month, FIR_Month_Name) added and verified. Column name Unit_id now consistent across Crime, Dispose, and Act tables — ready for composite key creation.

3.2 Step 2 — Create Composite Key & Deduplicate

What was done

A composite key column called Key was created across all three core tables — Crime, Dispose, and Act — by concatenating Fir_id and Unit_id with a hyphen separator:

```
# Create composite key on all three tables
Crime['Key']   = Crime['Fir_id']   + '-' + Crime['Unit_id']
Dispose['Key'] = Dispose['Fir_id'] + '-' + Dispose['Unit_id']
act['Key']     = act['Fir_id']     + '-' + act['Unit_id']

# Deduplicate crime to one row per FIR
crime_unique = Crime.drop_duplicates(subset=['Key']).copy()

print("Unique FIRs:", len(crime_unique))
print("Crime keys sample:", crime_unique['Key'].head(3).tolist())
```

```
Unique FIRs: 140585
Crime keys sample: ['FIR0001-UNIT001', 'FIR0009-UNIT001', 'FIR0022-UNIT001']
```

Why it was done

A single FIR number (Fir_id) alone is not sufficient to uniquely identify a case across the dataset, because the same FIR number can exist across different police stations (Unit_id). Using only Fir_id as a join key would cause incorrect cross-station matches. The composite key Fir_id + Unit_id ensures that each key maps to exactly one FIR at one specific police station which is the true unique identifier for a case in this system.

The Crime table also contains multiple rows per FIR because a single case can be registered under multiple IPC sections or act descriptions. If you join the Dispose table directly without deduplicating first, each FIR would appear multiple times in the result, causing all aggregate counts (Total Cases, Disposed, Pending) to be inflated. Deduplication collapses all rows for a given FIR into one, making the case counts accurate.

Result

Crime table reduced from **2,11,863 raw rows** to **1,40,585 unique FIRs** after deduplication on the composite key. This means approximately 71,278 rows were duplicates caused by multiple act section entries per FIR — not duplicate case registrations. The crime_unique DataFrame is now the single source of truth for all downstream joins and aggregations.

3.3 Step 3 — Join Dispose Table (Disposed vs Pending)

What was done

The Dispose table was left-joined onto crime_unique on the Key column, bringing in the Dispose_Date column for each FIR:

```
crime_unique = crime_unique.merge(
    Dispose[['Key', 'Dispose_Date']],
    on='Key',
    how='left'
)

# Create flags
crime_unique['Is_Disposed'] = crime_unique['Dispose_Date'].notnull().astype(int)
crime_unique['Is_Pending']  = (crime_unique['Dispose_Date'].isnull()).astype(int)

# Check
print("Total FIRs:", len(crime_unique))
print("Disposed:", crime_unique['Is_Disposed'].sum())
print("Pending:", crime_unique['Is_Pending'].sum())
```

```
Total FIRs: 140689
Disposed: 6125
Pending: 134564
```

After the join, two binary flag columns were created:

- **Is_Disposed = 1 if Dispose_Date is not null (a disposal record exists for this FIR)**
- **Is_Pending = 1 if Dispose_Date is null (no disposal record exists — case is still open)**

Why it was done

The raw Crime table has no status column. There is no field that directly says "this case is open" or "this case is closed." The only way to determine case status is to check whether a corresponding record exists in the Dispose table if a disposal date has been recorded, the case is closed; if not, it is still pending.

A left join was used specifically to preserve all 1,40,585 FIRs from crime_unique regardless of whether they have a match in the Dispose table. If an inner join had been used instead, all pending cases (those with no disposal record) would have been silently dropped from the dataset — the worst possible outcome for a system designed to track backlogs. The left join ensures that unmatched FIRs remain in the table with Dispose_Date = NaT, which then correctly gets flagged as Is_Pending = 1.

Result

Total FIRs	Disposed	Pending
1,40,688	12,935 (9.2%)	1,27,753 (90.8%)

All 1,40,585 FIRs retained after the join. The Is_Disposed and Is_Pending flag columns now exist on every row. From this join it was confirmed that approximately **91% of cases in the dataset are pending** — meaning only ~9% of FIRs registered between 2020 and 2025 have a recorded disposal. This single finding is the clearest validation that CRU-IAS is solving a real and urgent operational problem.

3.4 Step 4 — Join Act_Section Table (Crime Type)

What was done

The Act_Section table (act) contains multiple rows per FIR one row for each law section or act under which the case was registered. For example, a single FIR might be registered under both IPC 1860 and the IT Act 2000, resulting in two rows in the Act table for the same Key.

To keep one clean row per FIR in the master table, only the primary act per FIR was extracted using drop_duplicates(subset=['Key']) which keeps the first act record encountered for each unique Key and discards the rest:

```
act_grouped = act.groupby('Key')['Act_Description'].apply(lambda x: ', '.join(x)).reset_index()
```

This act table was then left-joined onto crime_unique on the Key column, bringing in Act_Description as the crime type label for each FIR. Any FIR with no matching act record received NaN for Act_Description, which was immediately filled with the string 'Unknown':

```
merged = crime_unique.merge(Dispose[['Key','Dispose_Date']], on='Key', how='left')
merged = merged.merge(act_grouped, on='Key', how='left')
merged
```

	District_Name	UnitName	FIRNo	Pin_Code	Direction_Distance	Beat	Village	FIR Type	FIR_Stage	CrimeGroup_Name	...	FIR_Date	FIR_Year	FIR_Month	FIR_Month
0	Bengaluru City	North CEN Crime PS	1.044320e+17	0	0	2019000001	YPR CENPS	Non-Heinous	Disposal	CYBER CRIME	...	2021-08-06 10:45:00	2021	8	
1	Bengaluru City	North CEN Crime PS	1.044320e+17	0	0	2019000001	YPR CENPS	Non-Heinous	Pending	CYBER CRIME	...	2021-08-20 10:00:00	2021	8	

Why it was done

The dashboard filter works on a single crime type per FIR. If all act sections per FIR were retained (by joining without deduplication), each FIR would appear multiple times in the merged table — once per act section — causing Total Cases, Disposed Cases, and Pending Cases to all be overcounted in the final aggregation.

Taking only the first act record per FIR gives the primary charge under which the case was registered, which is the most meaningful label for dashboard filtering and grouping. This is a deliberate simplification — it trades multi-label completeness for clean, accurate case counts. The alternative approach (joining all acts and concatenating them as a comma-separated string, which was also tried in Cell 14) was discarded because it creates too many unique combinations in the final groupby, making the dashboard filter impractical.

The fillna('Unknown') step ensures no row is lost from the aggregation due to a missing act label. A null Act_Description would cause that FIR to be excluded from any groupby operation, silently reducing the total case count.

Result

Act_Description joined successfully to all 1,40,585 FIRs. **1,814 FIRs** had no matching record in the Act table and were filled as 'Unknown'. The dominant crime type in the dataset is **IPC 1860** with 95,073 FIRs, confirming it as the primary law under which cases are registered across Karnataka police stations. The merged table merged_clean now has one row per FIR with all required fields: Key, UnitName, FIR_Year, FIR_Month, FIR_Month_Name, Act_Description, Is_Disposed, and Is_Pending.

3.5 Step 5 — Build Master Summary & Aggregate

What was done

With merged_clean containing one clean row per FIR and all required fields attached, the next step was to collapse this FIR-level table into a grouped summary that the dashboard can directly consume.

The aggregation was done using groupby on five dimensions — UnitName, FIR_Year, FIR_Month, FIR_Month_Name, and Act_Description — and three metrics were computed per group:

```
summary_final = merged.groupby(
    ['UnitName', 'FIR_Year', 'FIR_Month', 'FIR_Month_Name', 'Act_Description']
).agg(
    Total_Cases=('Key', 'count'),
    Disposed_Cases=('Is_Disposed', 'sum'),
    Pending_Cases=('Is_Pending', 'sum')
).reset_index()

summary_final['Pending_Percentage'] = (
    summary_final['Pending_Cases'] / summary_final['Total_Cases'] * 100
).round(2)

# Sort by year and month for clean ordering
summary_final = summary_final.sort_values(['FIR_Year', 'FIR_Month'])

# Save
summary_final.to_csv('cru_aggregated.csv', index=False)

print("Shape:", summary_final.shape)
print("Total cases:", summary_final['Total_Cases'].sum())
print("\nSample:")
print(summary_final.head())
```

```
Shape: (28992, 9)
Total cases: 139089
```

The result was sorted by FIR_Year and FIR_Month for clean chronological ordering, then saved as cru_aggregated.csv — the final output file that the HTML dashboard reads.

Aggregated Output Columns

Column	Type	Description
UnitName	Text	Police station name e.g. Ashoknagar PS
FIR_Year	Integer	Year the FIR was registered
FIR_Month	Integer	Month number (1–12)
FIR_Month_Name	Text	Month name e.g. January
Act_Description	Text	Primary law/act under which FIR was registered
Total_Cases	Integer	Total FIRs in this group
Disposed_Cases	Integer	Number of disposed FIRs in this group
Pending_Cases	Integer	Number of pending FIRs in this group
Pending_Percentage	Float	Pending cases as % of total (rounded to 2 decimals)

3.6 Step 6 — Data Quality Fixes & Export

Act name inconsistencies fixed

- 'NARCOTIC DRUGS & PSYCHOTROPIC SUBSTANCES ACT, 1985' merged with full name version
- 'INFORMATION TECHNOLOGY ACT 2000' extra space removed
- All Act_Description values stripped of leading/trailing whitespace

Final Output

File	Rows	Unique Act Types
cru_aggregated.csv	9,453	100

4. Analytics Dashboard — Phase 2

4.1 System Overview

The CRU-IAS Analytics Dashboard is a fully browser-based intelligence interface built in a single index.html file with embedded JavaScript. It has **no backend, no server, and no live network connection** which is exactly why it solves the VS Code fetch problem.

Instead of using **fetch()** to load a CSV from disk (which browsers block due to CORS security restrictions when opening HTML files locally), the dashboard uses a **file upload approach** the user manually uploads cru_aggregated.csv or the Excel equivalent through a drag-and-drop upload screen. The file is read entirely in the browser using the **SheetJS (xlsx) library**, parsed into a JavaScript array, and stored in memory. From that point, all filtering, aggregation, and chart rendering happens instantly in-memory with no further file reads.

The dashboard presents two dropdown filters **Police Unit** and **Year** and automatically renders four views upon selection: four KPI cards (Total Cases, Pending Cases, Solved Cases, Pending Rate), three Chart.js charts (Top 8

Act Types bar chart, Year-wise trend line for the selected unit, All Units comparison bar chart for the selected year), and a detailed breakdown table by Act Type with a pending rate status pill (High / Medium / Low).

4.2 File Structure

File	Purpose
index.html	Main dashboard page — layout, dropdowns, chart canvas
script.js	JavaScript logic — CSV loading, dropdown population, filtering, chart rendering
cru_aggregated.csv	Data source — pipeline output read by the dashboard

> **Note:** There is no separate script.js file. All logic lives inside index.html. This was a deliberate choice keeping everything in one file means the dashboard is fully portable. You can send or open a single file and it works completely without any folder structure dependency.

4.3 How It Works

Step 1 — File Upload & Parsing (Why not fetch()?)

When you open an HTML file directly in a browser (using file:// protocol, as VS Code Live Server does by default), the browser's CORS security policy blocks fetch() from reading local files from disk even if the CSV is sitting in the same folder. This is a browser security restriction, not a code bug.

The solution used in CRU-IAS is the FileReader + SheetJS approach. The user is presented with an upload screen where they select the cru_aggregated.csv or .xlsx file. JavaScript reads it using FileReader.readAsArrayBuffer(), then passes the raw bytes to the SheetJS XLSX.read() function. SheetJS parses the file whether it is a CSV or Excel format and converts it to a JSON array using XLSX.utils.sheet_to_json(). Each row becomes a JavaScript object with column names as keys. This entire process happens inside the browser with zero network requests.

```
const reader = new FileReader();
reader.readAsArrayBuffer(file);
reader.onload = function(e) {
        const wb = XLSX.read(new Uint8Array(e.target.result), { type: 'array' });
        const ws = wb.Sheets[wb.SheetNames[0]];
        rawData = XLSX.utils.sheet_to_json(ws, { defval: "" });
}
;
```

A cleaning step immediately follows all column keys and string values are trimmed of whitespace, BOM characters (\ufeff), and surrounding quotes to ensure consistent matching downstream.

Step 2 — Dropdown Population

After rawData is built, unique values for UnitName and FIR_Year are extracted using JavaScript's Set() object, sorted, and injected as <option> tags into the two dropdown <select> elements. The Grand Total Banner is also

populated at this point it shows dataset-wide totals (all units, all years) as a permanent reference: Total Cases, Total Pending, Total Solved, number of unique Units, and number of Years covered. This gives the user context before they start filtering.

Step 3 — Filtering & Analysis

Every time the user changes either dropdown, updateUI() fires. It filters rawData using .filter() to match the selected UnitName and FIR_Year, then reduces the filtered rows into four KPI values — Total Cases, Pending Cases, Solved Cases (Total minus Pending), and Pending Rate (pending ÷ total × 100). These four numbers are written directly into the KPI card elements on screen.

Step 4 — Chart Rendering

Three charts are rendered after every filter change, each destroyed and rebuilt cleanly using chart.destroy() before a new Chart() instance is created:

Top 8 Act Types (Horizontal Bar) groups the filtered rows by Act_Description, sums Total_Cases per act, takes the top 8 by volume, and renders a horizontal bar chart with 8 distinct colors. Act names longer than 30 characters are truncated with an ellipsis to fit the chart labels.

Year-wise Trend Line filters rawData for the selected unit across all years (not just the selected year), groups by FIR_Year, and renders a filled line chart showing how case volume at that police station has moved year over year from 2020 to 2025. The gradient fill under the line makes the trend visually clear.

All Units Comparison (Bar) filters rawData for the selected year across all units, takes the top 15 units by case volume, and renders a bar chart comparing them. The currently selected unit's bar is highlighted in orange (#e8923a) while all others appear in blue so the user can instantly see where their selected station ranks.

4.4 Challenges & Solutions

Challenge	Cause	Solution
Dropdowns not populating	Wrong CSV filename in fetch()	Changed crime_summary.csv to cru_aggregated.csv
Wrong column mapping	New FIR_Month column shifted all column positions	Updated col[] index in JS to match new CSV structure
Chart not rendering	Old chart not destroyed before creating new one	Added chart.destroy() before each new Chart() call
[Add your own challenge]	*[Cause]*	*[How you fixed it]*

5. Overall Challenges & Solutions

The filtered rows are grouped by Act_Description and summed into a table showing Total Cases, Pending Cases, and Pending Rate per act. Each row gets a color-coded status pill — **High** (red, >70% pending), **Medium** (amber, 40–70% pending), or **Low** (green, <40% pending). A small progress bar under Total Cases shows each act's volume relative to the highest-volume act in the filtered set.

#	Challenge	Observation	Action Taken
1	Duplicate FIR records	Crime table had 26,256 duplicate rows out of 2,11,863	Deduplicated on composite Key (Fir_id + Unit_id)
2	Multiple act sections per FIR	Act table had 3,49,029 rows for 1,38,771 unique FIRs	Took primary act per FIR using drop_duplicates on Key
3	Inconsistent act names	Same act written differently e.g. & vs and, double spaces	Standardized using .replace() and .str.strip()
4	Missing UnitName after dedup	Column dropped during early pipeline iterations	Rebuilt pipeline cleanly from import in single flow
5	fetch() not loading CSV in VS Code	Browsers block fetch() on file:// protocol due to CORS even for local files in the same folder	Switched to a FileReader + SheetJS file upload approach user uploads the file directly, no network request needed
6	Dropdowns not populating	Column name UnitName had a BOM character (\ufeff) prefix in some CSV exports, so r.UnitName returned undefined	Added a BOM-stripping step during key cleaning: .replace(/^\ufeff/, '') on all column names during parsing
7	Wrong column mapping	Early versions used positional col[index] access adding or reordering columns in the CSV broke all downstream references	Switched to named key access (r.UnitName, r.FIR_Year, r.Total_Cases) so column order in the CSV no longer matters
8	Chart not rendering on second filter change	Old Chart.js instance was not destroyed before creating a new one charts were stacking on the same canvas causing visual corruption	Added if (actChart) actChart.destroy() before every new Chart() call for all three chart instances
9	File supports both CSV and Excel	Pipeline output is CSV but some users may export as .xlsx	Used SheetJS (xlsx library) instead of manual CSV parsing — SheetJS handles both .csv, .xls, and .xlsx with the same code path

6. Next Phases

6.1 Phase 2 — Analytics Dashboard (In Progress)

The current dashboard answers one specific question per query select a station, year, and crime type, get three numbers and a bar chart. The next round of dashboard development will expand it significantly:

- Monthly trend line chart — show how Total, Disposed, and Pending cases move month by month across a selected year for a given station and crime type, so seasonal patterns become visible.
- Pending percentage gauge — a visual dial or progress bar showing the pending % for the selected filter, making the backlog severity immediately obvious without reading numbers.
- District-level aggregated view — currently the dashboard filters at police station level. A district rollup view would let senior officers see all stations within a district in one screen.

- Top 5 backlog stations — a ranked list or horizontal bar chart showing which police stations have the highest pending case count for a selected year and crime type, requiring no manual filtering.
- All stations / All years option — add an "All" default to each dropdown so the dashboard can show state-wide or year-wide totals without requiring a specific selection.

6.2 Phase 3 — Predictive Model

Once the dashboard phase is complete, the clean cru_aggregated.csv output becomes the training data for a predictive model. The goal of Phase 3 is to forecast crime volume specifically, to predict how many FIRs a given police station is likely to receive in a future month for a given crime type, based on the 2020–2025 historical pattern.

The model will be built in Python using scikit-learn. The planned approach is to start with a **Random Forest Regressor** as the baseline it handles the categorical features (UnitName, Act_Description) well after encoding and gives feature importance rankings that explain which variables drive crime volume most. If the monthly trend data shows strong seasonality, a **time series model (Prophet or ARIMA)** will be layered on top for station-level forecasting.

Outputs from the predictive model will be fed back into the dashboard as a new "Forecast" tab letting a station officer see not just the current backlog, but also how many new cases to expect next month so resources can be planned ahead of time.

6.3 Future Enhancements

- **Live data integration** — replace the static CSV with a database connection (PostgreSQL or SQLite) so the dashboard reflects new FIR registrations in near real-time without re-running the pipeline manually.
- **User authentication layer** — add login-based access control so each officer only sees data for their own station or district, which is essential for any real government deployment.
- **Export to PDF/Excel** — add a download button that lets the user export the current filtered view as a formatted report, making it usable in official meetings and review sessions without screenshots.
- **Mobile-responsive layout** — optimize the dashboard UI for tablet and mobile screens so field officers can access it without needing a desktop browser.
- **Automated pipeline scheduling** — use a scheduler (cron job or Apache Airflow) to run the Python pipeline automatically at regular intervals, so cru_aggregated.csv is always up to date without manual Jupyter execution.